Panel 9 architecture that Bean uses

Audience: developers and operators who need to understand how Panel 9 containers exist, how Bean drives them, how YouTube channel switches work inside one Google login, and what YouTube Focus actually strips.

Freshness (read this first). This write-up is from published local architecture docs on the build box, not a live inspection of any PC:

App	Docs / store version used here
Panel 9 (`panel-9`)	Architecture **2.3.28** (store latest listed). Local `package.json` may show **2.3.29** (Echo playbook URL only).
Bean (`panel9-bean`)	Architecture **1.1.7.9**. Note: some Bean architecture file headers still say "current 1.1.0" — trust the filename / store version.
YouTube Focus (`yt-focus`)	Architecture **1.4.0**

Live installed versions on a user's machine were **not** re-checked for this document. No secrets, cookies, tokens, emails, or partition paths from a live install are included.

---

1. Overview

Panel 9 is an Electron browser (Windows x64) with Chrome-style **profiles**, **account containers**, tabs, proxies / VPN hooks, and a Chrome-extension platform. Bean is a Panel 9 extension that likes YouTube comments from chosen containers and channels. YouTube Focus is a separate extension that makes watch pages light enough for Bean (especially "Comments only").

The important mental model:

```
Profile (on-disk user)
  └── Container (Electron persist session / cookie jar)
        └── Tab(s) (UI views bound to that container's session)
              └── YouTube channel(s) under that Google login
```

A **container** is not a Docker container. It is an isolated browser session partition.

---

2. What a container is, and how it exists

2.1 Definition

Inside a profile, a **container** is a named, coloured **isolated Electron session**. Its partition name looks like:

```
persist:p<profileId>-c<containerId>
```

That partition is the cookie jar, login world, and site storage for that account lane. The first container (`default`) is the profile's **Main** session.

2.2 What is isolated vs shared

Isolated per container

- Cookies and Google sign-in for that jar
- Local site storage for that session
- YouTube channel state belonging to that login
- Optional per-container proxy / Outside-VPN override

Shared across containers (same PC / profile world)

- The machine itself
- ExpressVPN as an app (unless Outside-VPN / proxy redirects a session)

- Device-emulation / Chrome-identity machinery Panel 9 applies to sessions
- Extension `chrome.storage.local` — Panel 9 **mirrors** it across sessions so Bean / Focus settings apply everywhere

2.3 Tabs, accounts, and channels are not the same thing

Concept	What it is
Container	Persistent Electron session partition (`persist:p…-c…`). The real isolation boundary.
Tab	A `WebContentsView` UI surface bound to one container's session. Closing a tab does not delete the container.
Google account	Detected *inside* a container (auth cookies + ListAccounts). Does not create a partition by itself.
YouTube channel	A channel / brand identity under one Google login. Multiple channels can live in **one** container.

Google's own multi-login cap is about **10 accounts per cookie jar**. Panel 9 therefore uses many containers so one profile can hold more than 10 Google accounts. Switching to another Google account usually means moving to another container. Switching brand channels under the same Google login stays in the same container.

2.4 Lifecycle and persistence

1. **Create** — container metadata is stored under the profile (`containers.json` and related ProfileData JSON). A new partition id is allocated.
2. **Use** — tabs open in that partition; Google sign-in cookies live there; OAuth popups stay attached to the opener's session.
3. **Survive restart** — because the partition is `persist:…` , logins survive app restarts. Optional session restore can reopen tabs with their container ids.
4. **Sign out** — erases that container's session data but keeps the container entry.
5. **Remove** — deletes the container (not offered for built-in Main), queues partition purge (`pendingDelete`), and purges data on next start (`purgeDeletedData`).
6. **Idle / light sessions** — idle containers can drop extension renderer weight while keeping cookies; opening a real browsing tab re-attaches extensions. This exists to reduce hang / CPU weight when many containers exist.

Gotcha: partition names are persistent identifiers. Renaming the partition scheme breaks existing logins.

Private windows are different: they use disposable / wiped partitions and are not Bean's normal lane.

---

3. How Bean selects and drives containers

Electron's extension subset has no full `chrome.tabs` UI API, so Bean cannot list containers or open tabs in them by itself. Panel 9 **≥ 2.3.9** adds a privileged bridge (`window.panel9ext`), allow-listed only for Bean's popup (store id / local id matching `panel9-bean`):

Bridge call	Purpose
`containers()`	Lists containers in the current profile window (id, name, color, signed-in hints). **No emails, cookies, or partition names.**
`openTabInContainer({containerId, url})`	Opens a desktop tab in that container (YouTube watch URL, or home URL for channel discovery). Rate-limited.
`closeTab({tabId})`	Closes **only** tabs this bridge opened. User tabs are refused.

3.1 One-tab work model

Bean's run planner works **container → channel → link**:

1. User ticks containers (and channels under them) and pastes comment links.
2. Bean writes a run record in mirrored `chrome.storage.local` .
3. The orchestrator opens **at most one Bean-owned tab at a time** in the next container.
4. The content script in that tab processes that container's queued items (with pacing).

5. When the container group is done (and optional channel restore finished), Bean closes that tab, then opens the next container's tab.

Without the bridge (older Panel 9), Bean degrades to the **active tab's container** only.

Bean also expects **YouTube Focus "Comments only"** to be on. It does not read Focus's storage directly; it reads Focus's DOM markers on `<html>` (see §5).

VPN rotation (when configured) is coordinated around likes / container opens via a narrow Panel 9 VPN bridge. A failed / unconfirmed VPN switch means **no like** for that item.

---

4. Channel switches

There are two different "switches." Mixing them up is the usual source of confusion.

4.1 Different Google account → different container

Panel 9's account switcher moves the current tab to the container that holds that account (same URL, `?authuser=` when needed). Bean does the multi-account case by opening a tab **in each ticked container**. Same Google login does **not** require a new container.

4.2 Brand / secondary YouTube channel → same container

Several YouTube channels can belong to one Google account. Bean handles that **inside one container**:

1. **Refresh channels** opens a short-lived home tab in the container and calls YouTube's own same-origin `GET /getAccountSwitcherEndpoint` (the call the page makes for Switch account).
2. Bean caches channel id / name / handle / avatar only — not cookies or tokens.
3. Before liking on a non-active channel, Bean navigates to YouTube's own channel `signinUrl` (with `pageid` / `next=` pointing back at the watch URL carrying Bean's run tokens).
4. After verification gates pass, it likes; optionally it restores the original channel when the container's work is done.

Identity gates (page session from ytcfg + account-menu active entry, optional masthead check) must agree before a click. If the switch cannot be verified → `switch-failed`, **no like**.

Confidence note: switcher JSON shape and sign-in redirects are documented from public / mock-verified sources. Live YouTube experiments can still drift; treat edge cases as not fully proven against every real account.

Panel 9's own 2.3.22 investigation concluded that, in container sessions, main-frame and fetch requests are treated the same for cookies/headers/caching. Remaining "main frame shows old channel / fetch shows new" reports were attributed to YouTube's own handling, not Panel 9 rewriting cookies.

---

5. YouTube Focus stripping (what Bean depends on)

YouTube Focus does **not** rewrite Google auth, steal cookies, change extension permissions for login, or call privileged YouTube owner APIs. It changes what the page loads and shows.

5.1 Switches and markers

Focus stores switches in `chrome.storage.local` and mirrors them onto `<html>` attributes such as:

- `data-ytf-comments` — Comments only switch is ON (Bean's gate)
- `data-ytf-focus` — Comments only **and** this is a watch page
- related markers for stop-videos / hide-suggested / account chain / sign-out hiding

Bean re-checks these markers in the page before liking.

5.2 What "stripping" means

Mechanism	Effect

CSS / DOM	Comments-only hides player, chrome, sidebar; keeps comments usable. Hide Sign out removes logout entries from menus (account/channel rows and Add account stay).
Page-world network/media hooks	Can stop `googlevideo` playback requests and media `play()` while stop / comments-only is effective.
Lean load (Focus 1.4.0 + Panel 9 2.3.21+)	In Chrome, declarativeNetRequest can block player/ads. In Panel 9, main-process `ytf-lean` cancels equivalent requests on watch pages while Comments only is on, and clears YouTube service-worker / cache storage so the player is not served from cache. Comment / switcher / sign-in paths stay allowed.

Turning Comments only off restores normal YouTube behaviour (some blocked pages reload once so the player can start cleanly).

5.3 Performance consequences

- Cuts bytes, player work, ads/telemetry, and visual noise — useful so Bean tabs do not drag a full YouTube UI.
- Docs note that **time-to-clickable-like** may not improve much, because large YouTube app JS/CSS still load for comments.
- Many containers still cost session weight, tab churn, and optional VPN rotations. Panel 9 light-session release and Bean's one-tab rule exist specifically to keep long runs from hanging the UI.

---

6. Limits and honesty checklist

Documented / architecture-verified

- Container = `persist:p…-c…` session partition
- Tabs are UI; containers persist independently
- Bean bridge + one-tab orchestration
- Same-container YouTube channel switch via YouTube's own switcher / signinUrl
- Focus stripping is DOM/CSS/network lean, not auth rewrite
- Storage mirroring across sessions
- Idle / light session release for weight

Not claimed by this document

- Exact version currently installed on a live PC
- Every real-YouTube channel-switch edge case
- That Focus makes likes latency-free
- Any live hang measurement for a specific machine

Operational reminder: Panel 9 store / control defaults moved toward `axionswap.com` names in 2.3.28 docs; older installs pointing at a held old hostname may fail to auto-update until pointed at the new API. That is infrastructure, not container semantics.

Appendix A — Optional source references

Paths are on the build / docs box used to write this file:

- `/workspace/appstore/apps/panel-9/architecture/2.3.28.md` — profiles/containers, extensions, Bean bridge, lean filter, channel-switch investigation notes
- `/workspace/p9-control/ARCHITECTURE.md` — same material in the Panel 9 tree (includes 2.3.21–2.3.22 lean / navtrace notes)
- `/workspace/appstore/apps/panel9-bean/architecture/1.1.7.9.md` — Bean run model, bridge, channel discovery/switch, one-tab rule
- `/workspace/appstore/apps/yt-focus/architecture/1.4.0.md` — Focus switches, DOM markers, lean rules / Panel 9 lean counterpart

Store manifests listing latest published versions:

- `/workspace/appstore/apps/panel-9/manifest.json` → 2.3.28
- `/workspace/appstore/apps/panel9-bean/manifest.json` → 1.1.7.9

- `/workspace/appstore/apps/yt-focus/manifest.json` → 1.4.0

Standalone deliverable for sharing. No credentials. No live machine state.